VIA501 2026-Fall Homework Assignment # 1

Remarks:
Write the code yourself.

For each problem write your code in the function format and give the names of the functions as problem numbers, for example for the solution of problem1:

```
def problem1(input):
  return something
```

Put the codes for all problems into one file (jupter notebook file). The notebook file should definitely contain the outputs of the functions, if applicable. Sample solution file (sample_solution.ipynb) is given to you to show how to organize your solutions.
Also write your name and student number inside the jupyter notebook as well.

Give as much as documentation for your script using comments.

Note1: For the following problems, **do not** use libraries like Numpy or PANDAS. You are allowed to use only built-in python modules (os, sys, time, collections etc.)
Note2: If your homework solution file has problems in structure you can lose up to ***20*** points!! For example, if you didn't write solutions in a function format or if you did not arrange input arguments properly you may lose points.

Problem 1 (15 Points).

A store charges $12 per item if you buy less than 10 items. If you buy between 10 and 99 items, the cost is $10 per item. If you buy 100 or more items, the cost is $7 per item. Write a program that takes how many items are bought as an input and prints the total cost.

```
problem1(20)
```

```
total cost is $200
```

Problem 2 (20 Points).

Write a program that generates a list of 20 random numbers between 1 and 100. When generating random number use random seed as 18.

(a) Print the list.
(b) Print the sorted list (sort list in the descending order)
(c) Print the average of the elements in the list.
(d) Print how many even numbers are in the list.
(e) Print the largest and smallest values in the list.
(f) Print the second largest and second smallest entries in the list

Since we use the same random seed number, your output should be exactly like this:

```
problem2()
```

```
the numbers in the list:
 [24, 16, 85, 58, 43, 31, 26, 63, 81, 64, 24, 62, 38, 59, 34, 26, 33, 89, 16, 42]
the sorted list:
 [89, 85, 81, 64, 63, 62, 59, 58, 43, 42, 38, 34, 33, 31, 26, 26, 24, 24, 16, 16]
the average of the list: 45.7
the number of even-numbers is : 12
the largest element of the list: 89
the smallest element of the list: 16
the second smallest element of the list: 24
the second largest element of the list: 85
```

Problem 3 (20 Points).

Randomly generate a 6 x 6 list that has exactly 12 ones placed in random locations in the list. The rest of the entries should be zeroes. Note you can visualize the 2D list using display function.

```
problem3()
```

```
[[0, 1, 0, 1, 0, 1],
 [1, 0, 1, 0, 0, 1],
 [0, 1, 0, 1, 1, 0],
 [0, 0, 0, 0, 0, 0],
 [1, 0, 0, 1, 0, 0],
 [0, 0, 0, 1, 0, 0]]
```

Problem 4 (20 Points).

You are given log file (output23) from a bioinformatics scientific program. This output contains results for some analyses for multiple ligands (molecules). You are requested to extract information from this log file.

For a given ligand it first loads the data as given below.
Our ligand below is "**hmnrtd**".
After loading the program does some calculations. A screenshot of the first few lines of the output23 file is given below.

```
Computing grids on a055.uhem.itu.edu.tr a Linux-4.18.0-305.el8.x86_64-x86_64-with-redhat
Date Sat Oct 28 12:18:18 2023

loading receptor: maestro_prepared_Ungrouped.pdbqt
loading ligand: hmnrtd.pdbqt

set box using user
    Box center:    29.420    36.594    41.448
    Box length:    36.000    29.520    28.500
    Box size  :        96        80        76
    padding   :     0.000
    spacing   :     0.375

identifying pockets using AutoSite ....
Scanning at: -0.36 -0.792 -0.6
clust.| Energy| # of |Rad. of | ,energy |   bns    |score   |
number|       |points|gyration|per vol.|buriedness|v*b^2/rg|
------+-------+------+--------+--------+----------+--------|
    1   -29.03    54    2.42    -0.54     0.797     14.16
Scanning at: -0.315 -0.693 -0.525
clust.| Energy| # of |Rad. of | energy |   bns    |score   |
number|       |points|gyration|per vol.|buriedness|v*b^2/rg|
------+-------+------+--------+--------+----------+--------|
    1   -39.99    76    2.61    -0.53     0.782     17.79
Scanning at: -0.27 -0.594 -0.45
clust.| Energy| # of |Rad. of | energy |   bns    |score   |
number|       |points|gyration|per vol.|buriedness|v*b^2/rg|
------+-------+------+--------+--------+----------+--------|
    1   -65.86   128    3.22    -0.51     0.748     22.29
    2   -45.19    87    3.33    -0.52     0.753     14.82
    3   -23.81    53    2.51    -0.45     0.701     10.39
    4   -23.43    51    2.28    -0.46     0.525      6.18
```

When you continue scrolling you should encounter some lines like given below.

For the ligand "hmnrtd" we should extract the **performed affinity (kcal/mol)** information for the first **docking mode** which is highlighted as below.

You should write this information into a csv file as "hmnrtd,-11.2"

```
**********************************************
**************************************************
Docking performed in 157.65 seconds, i.e. 0 hours 02 minutes 37.647747 seconds
mode |  affinity  | clust. | ref. | clust. | rmsd | best | energy | best |
     | (kcal/mol) | rmsd   | rmsd |  size  | avg. | rmsd |  avg.  | run  |
-----+------------+--------+------+--------+------+------+--------+------+
   1        -11.2     0.0    52.6      32    52.4  51.5   -10.4    1268
   2        -11.0    14.1    59.3      16    59.2  59.0   -10.4     468
   3        -11.0     4.0    50.9      34    51.3  50.4    -9.0     954
   4        -10.5    14.0    58.5      25    58.8  58.3    -9.1     470
   5        -10.4    25.7    68.0      20    68.4  68.0    -9.7    2357
   6        -10.4     7.4    55.4      31    55.4  54.6    -8.7     944
   7        -10.4    14.3    59.6       2    59.5  59.5   -10.1     473
   8        -10.3    26.6    68.4      17    68.2  67.3    -8.9    2351
   9        -10.2    25.1    67.4      10    67.4  67.2    -9.7    1108
  10        -10.2    23.8    64.7       8    64.7  64.6    -9.8    1103
  11        -10.2    15.8    65.0      12    65.0  64.9    -9.7    2054
  12        -10.0    14.3    60.5       1      NA    NA    2056
  13        -10.0     5.4    54.1      30    54.3  53.0    -8.9     833
  14         -9.9    17.6    63.8      16    63.8  63.5    -9.2    1915
  15         -9.9     4.2    51.8      10    51.9  51.7    -9.5     152
  16         -9.9    15.8    62.4       6    62.4  62.4    -9.7     256
  17         -9.8     7.7    54.7       9    54.9  54.4    -9.3     703
```

The output23 file contains results for many ligands. You should extract above mentioned result and write into a csv file.

You should write a function that takes the name of the log file and the name of the result file as given below:

problem4("output23","result.csv")

Your function should write results.csv as given below.

result.csv

```
sequence,docking_score
hmnrtd,-11.2
nlhcnp,-9.6
ryfkhe,-12.5
iglely,-13.2
hncnmc,-10.2
ncyfps,-9.2
rwinvk,-12.2
csfedg,-11.9
ekmscv,-10.6
mchnka,-10.1
spmdvn,-9.9
awvptd,-11.1
fhwscp,-10.2
dhkefs,-12.8
mtswtf,-10.8
riiyqr,-10.5
cnlhpi,-9.6
```

Note that there are 499 ligand in this log file. Thus in the result.csv file there should be 500 lines including the header.

Problem 5 (20 Points).

Remember the movie rating example presented in week1. We merged the contents of movies.dat, ratings.dat and users.dat files into one dataframe table using PANDAS library. Here, in this problem you are asked do the same operation **without** using PANDAS library. To get the movie rating data go to this link: https://grouplens.org/datasets/movielens/ and download the ml-1m.zip file.

Write a function that takes three files as input and writes a text file (named as "merged.dat") for merged data. Merged data should have the following order.

user_id movie_id rating timestamp gender age occupation zip title genres Year

Note that we added year information as an extra column!
The input arguments for the functions should be exactly in the following order:

Merged.dat file should be like this:

```
problem5("movies.dat","users.dat","ratings.dat","merged.dat")
```

merged.dat — homework1

```
1::1193::5::978300760::F::1::10::48067::One Flew Over the Cuckoo's Nest ::Drama::1975
1::661::3::978302109::F::1::10::48067::James and the Giant Peach ::Animation|Children's|Musical::1996
1::914::3::978301968::F::1::10::48067::My Fair Lady ::Musical|Romance::1964
1::3408::4::978300275::F::1::10::48067::Erin Brockovich ::Drama::2000
1::2355::5::978824291::F::1::10::48067::Bug's Life, A ::Animation|Children's|Comedy::1998
1::1197::3::978302268::F::1::10::48067::Princess Bride, The ::Action|Adventure|Comedy|Romance::1987
1::1287::5::978302039::F::1::10::48067::Ben-Hur ::Action|Adventure|Drama::1959
1::2804::5::978300719::F::1::10::48067::Christmas Story, A ::Comedy|Drama::1983
1::594::4::978302268::F::1::10::48067::Snow White and the Seven Dwarfs ::Animation|Children's|Musical::1937
1::919::4::978301368::F::1::10::48067::Wizard of Oz, The ::Adventure|Children's|Drama|Musical::1939
1::595::5::978824268::F::1::10::48067::Beauty and the Beast ::Animation|Children's|Musical::1991
1::938::4::978301752::F::1::10::48067::Gigi ::Musical::1958
1::2398::4::978302281::F::1::10::48067::Miracle on 34th Street ::Drama::1947
1::2918::4::978302124::F::1::10::48067::Ferris Bueller's Day Off ::Comedy::1986
```

If a function like problem5 requires a file or folder write your function work appropritately independently from the path provided by the user.

For example, if the required files for problem5 are in a folder named "data-files" in my computer I should be able to run the function for problem5 like this:

```
problem5("data-files/movies.dat","data-files/users.dat","data-files/ratings.dat","merged.dat")
```

Also, input file names in the problem5 function should be in the same order as given above.

Note that the solution for this problem should not take more than **one minute**!

Problem 6 (5 Points).

In this problem, write a function that returns the number of hours you spent on this homework and difficulty level of the homework. Difficulty level should be between 1 and 5 referring very easy, easy, medium, difficult, very difficult respectively. The function should write the output into a file named "homework1_survey.txt".

